



P01-05 · T11–T12 aura-jobs — Job Graph, Leases, Heartbeats & Crash Resume



This is the crate that makes R2 true: **every run is resumable**. The rule it enforces is that no work is ever "in flight" only in memory. If a task is running, a row says so, and that row expires. Kill the process at any instant and the next launch knows exactly what was done, what was half-done, and what to redo.

1. `crates/aura-jobs/src/contract/graph.rs` — FROZEN

```
///! FROZEN CONTRACT. Every phase from 02 to 30 enqueues work
through this
///! contract and nothing else. Changing TaskId's format inval
idates every
///! resumable run in the field, so it is frozen hardest of al
l.
```

```
use std::fmt;
```

```
use aura_core::{AuraError, ProjectId, RunId};
use serde::{Deserialize, Serialize};
use time::OffsetDateTime;
```

```
/// Deterministic identity of a unit of work: `{stage}:{subje
```

```

ct}:{stage_version}`.
///
/// Deterministic, not random, and that is the whole design.
/// Enqueueing the same
/// logical work twice produces the same TaskId, so `INSERT O
R IGNORE` makes
/// enqueue idempotent for free. A resumed run re-enqueues ev
everything it
/// believes is needed and the already-finished tasks simply
collide.
///
/// stage_version is part of the identity so that shipping a
better version of a
/// stage invalidates exactly the affected work and nothing e
lse. Bumping
/// `thumbnail` from v1 to v2 re-runs thumbnails without touc
hing hashes.
#[derive(Debug, Clone, PartialEq, Eq, PartialOrd, Ord, Hash,
Serialize, Deserialize)]
pub struct TaskId(String);

impl TaskId {
    /// `subject` is a typed id rendered as text: a PhotoId,
    FileId or ImportId.
    /// Colons are forbidden in subjects so the format stays
    unambiguously
    /// parseable in both directions.
    pub fn new(stage: &str, subject: &str, stage_version: u3
2) -> Result<Self, AuraError> {
        if stage.is_empty() || stage.contains(':') {
            return Err(crate::errors::bad_task_id("stage must
be non-empty and contain no colon"));
        }
        if subject.is_empty() || subject.contains(':') {
            return Err(crate::errors::bad_task_id("subject mu
st be non-empty and contain no colon"));
        }
    }
}

```

```

    }
    Ok(Self(format!("{stage}:{subject}:{stage_version}
n}"))))
}

#[must_use]
pub fn as_str(&self) -> &str { &self.0 }

#[must_use]
pub fn stage(&self) -> &str {
    self.0.split(':').next().unwrap_or("") // ALLOW-BAN
NED: constructor guarantees 3 parts
}
}

impl fmt::Display for TaskId {
    fn fmt(&self, f: &mut fmt::Formatter<'_>) -> fmt::Result
    { f.write_str(&self.0) }
}

/// The only legal states, and the only legal transitions.
///
/// ```text
/// Pending —deps satisfied→ Ready —claim→ Running —
ok→ Succeeded
///      |                               |                               |
///      |                               |                               |—err,
attempts < max→ Ready
///      |                               |                               |—err,
attempts = max→ Failed
///      |                               |                               |—item
-level err→ Quarantined
///      |                               |                               |—leas
e expired→ Ready
///      |—dependency failed→ Skipped
/// ```

```

```

#[derive(Debug, Clone, Copy, PartialEq, Eq, Serialize, Deserialize)]
#[serde(rename_all = "snake_case")]
pub enum TaskState {
    /// Created, dependencies not yet satisfied.
    Pending,
    /// All dependencies succeeded. Claimable.
    Ready,
    /// Claimed by a worker holding a live lease.
    Running,
    /// Terminal, good.
    Succeeded,
    /// Terminal, bad: retries exhausted. Blocks dependents.
    Failed,
    /// Terminal, bad, but expected: this specific item cannot be processed.
    /// Does not block the run, only this subject's downstream work.
    Quarantined,
    /// Terminal, neutral: a dependency failed, so this was never attempted.
    Skipped,
}

impl TaskState {
    #[must_use]
    pub const fn is_terminal(self) -> bool {
        matches!(self, Self::Succeeded | Self::Failed | Self::Quarantined | Self::Skipped)
    }

    /// The single source of truth for legal transitions. The store calls this
    /// before every write, so an illegal transition is impossible rather than
    /// merely discouraged.

```

```

#[must_use]
pub const fn can_transition_to(self, next: Self) -> bool
{
    match (self, next) {
        (Self::Pending, Self::Ready | Self::Skipped)
=> true,
        (Self::Ready, Self::Running | Self::Skipped)
=> true,
        (Self::Running, Self::Succeeded | Self::Failed
          | Self::Quarantined | Self::Ready)
=> true,
        // Re-running a terminal task requires an explicit
t reset, which
        // deletes and re-creates the row with a new run_
id.
        _ => false,
    }
}

/// What this task will cost. Used by the scheduler to avoid
oversubscribing
/// RAM and VRAM. Estimates, honestly labelled as estimates.
#[derive(Debug, Clone, Copy, Default, PartialEq, Eq, Serializ
e, Deserialize)]
pub struct CostHint {
    pub cpu_ms: u32,
    pub gpu_ms: u32,
    pub ram_mb: u32,
    pub vram_mb: u32,
}

/// A time-boxed claim on a task.
///
/// TTL 90 s, heartbeat every 30 s: three missed heartbeats b
efore reclamation.

```

```

/// One missed heartbeat would reclaim work from a machine th
at merely swapped
/// or hit a long GC pause, and duplicated GPU work is expens
ive.
#[derive(Debug, Clone, PartialEq, Eq, Serialize, Deserializ
e)]
pub struct Lease {
    pub task_id: TaskId,
    pub owner: WorkerId,
    pub expires_at: OffsetDateTime,
    pub heartbeat_at: OffsetDateTime,
}

pub const LEASE_TTL_SECONDS: i64 = 90;
pub const HEARTBEAT_INTERVAL_SECONDS: i64 = 30;
pub const DEFAULT_MAX_ATTEMPTS: u32 = 3;

/// Identifies one worker within one process run. Regenerated
on every launch,
/// so a lease held by a dead process can never be mistaken f
or a live one
/// after a restart, even if the OS reuses the process id.
#[derive(Debug, Clone, PartialEq, Eq, Serialize, Deserializ
e)]
pub struct WorkerId(pub String);

/// A task as the scheduler sees it.
#[derive(Debug, Clone, Serialize, Deserialize)]
pub struct Task {
    pub task_id: TaskId,
    pub project_id: ProjectId,
    pub run_id: RunId,
    pub stage: String,
    pub subject_id: String,
    pub stage_version: u32,
    pub state: TaskState,

```

```

    pub priority: i32,
    pub attempts: u32,
    pub max_attempts: u32,
    pub cost: CostHint,
    pub error_code: Option<String>,
}

/// A task plus its dependencies, as submitted.
#[derive(Debug, Clone, Serialize, Deserialize)]
pub struct TaskSpec {
    pub task_id: TaskId,
    pub stage: String,
    pub subject_id: String,
    pub stage_version: u32,
    pub priority: i32,
    pub cost: CostHint,
    pub max_attempts: u32,
    /// Must already exist or be present in the same enqueue
    batch. A dependency
    /// on an unknown task is a programming error and is rejected
    at enqueue,
    /// not discovered at 3 a.m. as a stuck run.
    pub depends_on: Vec<TaskId>,
}

```

Why **Quarantined** is separate from **Failed**

One corrupt RAW out of 4,000 must not fail a wedding. **Failed** means "the machinery broke, stop and tell someone"; **Quarantined** means "this specific photo cannot be processed, carry on with the other 3,999." They have different severities, different UI, different runbooks, and — critically — different effects on dependents: a **Failed** task marks its dependents **Skipped** and raises the run as unhealthy, while a **Quarantined** task only removes that one photo from the rest of the pipeline.

2. Store — crates/aura-jobs/src/store.rs

```
use aura_catalog::Catalog;
use aura_core::{AuraResult, RunId};

use crate::contract::graph::*;

/// Enqueue is idempotent by construction: TaskId is deterministic and we use
/// INSERT OR IGNORE. Calling this twice with the same specs is a no-op, which
/// is exactly what resume needs.
pub fn enqueue(cat: &Catalog, project: ProjectId, run: RunId, specs: &[TaskSpec])
    -> AuraResult<usize>
{
    // Validate the whole batch before writing anything: a batch that is
    // partially valid is a batch that will strand a run.
    let known: std::collections::BTreeSet<&str> =
        specs.iter().map(|s| s.task_id.as_str()).collect();
    for spec in specs {
        for dep in &spec.depends_on {
            if dep == &spec.task_id {
                return Err(crate::errors::cyclic_dependency(spec.task_id.as_str()));
            }
            if !known.contains(dep.as_str()) && !cat.task_exists(dep)? {
                return Err(crate::errors::unknown_dependency(dep.as_str()));
            }
        }
    }
}
```



```

cat.writer().transact({
    let specs = specs.to_vec();
    move |tx| {
        let now = /* clock */ crate::now(tx)?;
        let mut inserted = 0usize;
        for spec in &specs {
            // A task with no dependencies is immediately
Ready. One with
            // dependencies starts Pending and is promote
d by promote_ready.
            let state = if spec.depends_on.is_empty() { T
askState::Ready } else { TaskState::Pending };
            let n = tx.execute(
                "INSERT OR IGNORE INTO task (task_id, pro
ject_id, run_id, stage, subject_id,
                stage_version, state, priority, attem
pts, max_attempts,
                cost_cpu_ms, cost_gpu_ms, cost_ram_m
b, cost_vram_mb, created_at, updated_at)
                VALUES (?1,?2,?3,?4,?5,?6,?7,?8,0,?9,?1
0,?11,?12,?13,?14,?14)",
                rusqlite::params![
                    spec.task_id.as_str(), project.to_str
ing(), run.to_string(),
                    spec.stage, spec.subject_id, spec.sta
ge_version,
                    state_str(state), spec.priority, spe
c.max_attempts,
                    spec.cost.cpu_ms, spec.cost.gpu_ms, s
pec.cost.ram_mb, spec.cost.vram_mb,
                    now,
                    ],
                ).map_err(crate::errors::store_write)?;
            inserted += n;

            for dep in &spec.depends_on {

```

```

        tx.execute(
            "INSERT OR IGNORE INTO task_dep (task
_id, depends_on) VALUES (?1, ?2)",
            rusqlite::params![spec.task_id.as_str
(), dep.as_str()],
            ).map_err(crate::errors::store_write)?;
        }
    }
    Ok(inserted)
}
})
}

```

/// Claim the highest-priority ready task and take a lease, i
n ONE statement.

///

/// The single-statement form is not a micro-optimisation: SE
LECT-then-UPDATE

/// would let two workers select the same row. Even with our
single writer this

/// stays atomic against a future multi-process design, and i
t costs nothing.

pub fn claim_next(cat: &Catalog, project: ProjectId, worker:
&WorkerId, budget: CostHint)

-> AuraResult<Option<Task>>

{

cat.writer().transact({

let worker = worker.clone();

move |tx| {

let now = crate::now(tx)?;

let expires = crate::now_plus(tx, LEASE_TTL_SECON
DS)?;

let mut stmt = tx.prepare_cached(

"UPDATE task SET state = 'running', lease_ow
ner = ?1, lease_expires_at = ?2,

heartbeat_at = ?2, attempts = attempt

```

s + 1, updated_at = ?3
        WHERE task_id = (
            SELECT task_id FROM task
            WHERE project_id = ?4 AND state = 'ready'

            AND cost_ram_mb <= ?5
            AND cost_vram_mb <= ?6
            ORDER BY priority ASC, created_at ASC, task_id ASC

            LIMIT 1
        )
        RETURNING task_id, stage, subject_id, stage_version, attempts, max_attempts,
            priority, cost_cpu_ms, cost_gpu_ms, cost_ram_mb, cost_vram_mb, run_id"
    ).map_err(crate::errors::store_write)?;
    // ORDER BY ends with task_id so that two tasks created in the same
    // millisecond are still claimed in a deterministic order.

    let mut rows = stmt.query(rusqlite::params![
        worker.0, expires, now, project.to_string(),
        budget.ram_mb, budget.vram_mb
    ]).map_err(crate::errors::store_write)?;
    match rows.next().map_err(crate::errors::store_write)? {
        Some(row) => Ok(Some(crate::store::row_to_task(row, project)?)),
        None => Ok(None),
    }
}

})

}

}

/// Extend a lease. Returns false if the lease was already re
claimed, in which

```

```

/// case the worker must abandon its work immediately rather
/// than write a result
/// for a task someone else now owns.
pub fn heartbeat(cat: &Catalog, task: &TaskId, worker: &Worker
rId) -> AuraResult<bool> {
    let updated = cat.writer().transact({
        let (task, worker) = (task.clone(), worker.clone());
        move |tx| {
            let now = crate::now(tx)?;
            let expires = crate::now_plus(tx, LEASE_TTL_SECON
DS)?;

            tx.execute(
                "UPDATE task SET heartbeat_at = ?1, lease_exp
ires_at = ?2, updated_at = ?1
                WHERE task_id = ?3 AND lease_owner = ?4 AND
state = 'running'",
                rusqlite::params![now, expires, task.as_str
(), worker.0],
                ).map_err(crate::errors::store_write)
            }
        })?;
    Ok(updated == 1)
}

/// Complete a task. The state machine is checked, not assume
d.
pub fn complete(cat: &Catalog, task: &TaskId, worker: &Worker
Id, outcome: Outcome)
-> AuraResult<()>
{
    cat.writer().transact({
        let (task, worker) = (task.clone(), worker.clone());
        move |tx| {
            let current = crate::store::read_state(tx, &tas
k)?;

            let next = outcome.next_state(current);

```

```

        if !current.can_transition_to(next) {
            return Err(crate::errors::illegal_transition
(task.as_str(), current, next));
        }
        let now = crate::now(tx)?;
        let n = tx.execute(
            "UPDATE task SET state = ?1, error_code = ?2,
error_detail = ?3,
                lease_owner = NULL, lease_expires_at
= NULL,
                updated_at = ?4, finished_at = CASE W
HEN ?5 THEN ?4 ELSE NULL END
                WHERE task_id = ?6 AND lease_owner = ?7",
            rusqlite::params![state_str(next), outcome.co
de(), outcome.detail(), now,
                next.is_terminal(), task.as
_str(), worker.0],
        ).map_err(crate::errors::store_write)?;
        if n == 0 {
            // Our lease was reclaimed while we were work
ing. Do not write.
            return Err(crate::errors::lease_lost(task.as_
str()));
        }
        // Promoting dependents in the SAME transaction i
s what makes the
        // graph safe across a crash: a task can never be
Succeeded while
        // its dependents are still stuck in Pending.
        crate::store::promote_ready(tx, &task)?;
        if next == TaskState::Failed {
            crate::store::skip_dependents(tx, &task)?;
        }
        Ok(())
    }
}

```

```

    })
}

```

promote_ready and skip_dependents

```

-- promote_ready: any Pending task all of whose dependencies
are Succeeded
-- becomes Ready. Run after every successful completion, scop
ed to the
-- dependents of the task that just finished so the cost is 0
(fan-out), not
-- O(graph).
UPDATE task SET state = 'ready', updated_at = :now
WHERE state = 'pending'
  AND task_id IN (SELECT task_id FROM task_dep WHERE depends_
on = :finished)
  AND NOT EXISTS (
    SELECT 1 FROM task_dep d JOIN task t ON t.task_id = d.dep
ends_on
    WHERE d.task_id = task.task_id AND t.state <> 'succeeded'
  );

-- skip_dependents: transitively mark everything downstream o
f a Failed task as
-- Skipped. Recursive CTE, bounded by the graph, so a diamond
dependency is
-- visited once.
WITH RECURSIVE downstream(task_id) AS (
  SELECT task_id FROM task_dep WHERE depends_on = :failed
  UNION
  SELECT d.task_id FROM task_dep d JOIN downstream ds ON d.de
pends_on = ds.task_id
)
UPDATE task SET state = 'skipped', updated_at = :now

```

```
WHERE task_id IN (SELECT task_id FROM downstream)
AND state IN ('pending', 'ready');
```

3. Reclamation and resume — `crates/aura-jobs/src/reclaim.rs`

```
use aura_catalog::Catalog;
use aura_core::AuraResult;

use crate::contract::graph::*;

/// Reclaim tasks whose lease expired. Called on startup and
/// then every 15 s.
///
/// On startup this is the entirety of crash recovery: any task that was
/// Running when the process died has an expired – or soon to
/// expire – lease and
/// returns to Ready. There is no separate "recovery mode", no
/// journal replay,
/// and no code path that only executes after a crash. The code that
/// recovers is
/// the code that runs every fifteen seconds in normal operation,
/// which is why
/// it is actually tested.
pub fn reclaim_expired(cat: &Catalog) -> AuraResult<u64> {
    let n = cat.writer().transact(move |tx| {
        let now = crate::now(tx)?;
        // attempts was already incremented at claim time, so
        // a task killed
        // three times lands in Failed rather than looping forever.
        tx.execute(
            "UPDATE task
            SET state = CASE WHEN attempts >= max_attempt
```

```

s THEN 'failed' ELSE 'ready' END,
        error_code = CASE WHEN attempts >= max_at
tempts
                                THEN 'AURA-JOB-7002' EL
SE 'AURA-JOB-7001' END,
        lease_owner = NULL, lease_expires_at = NU
LL, updated_at = ?1
        WHERE state = 'running' AND lease_expires_at IS
NOT NULL
        AND datetime(lease_expires_at) < datetime(?
1)",
        rusqlite::params![now],
    ).map_err(crate::errors::store_write)
    )?;
    if n > 0 {
        tracing::warn!(target: "jobs", reclaimed = n, "reclai
med expired leases");
    }
    Ok(u64::try_from(n).unwrap_or(0))
}

```

```

/// Called once, immediately after the catalog opens, before
any worker starts.
///
/// Startup reclaim ignores lease expiry entirely: every Work
erId is new this
/// process, so any Running row is by definition orphaned. Wa
iting 90 s for
/// leases to lapse would make our 2 s resume budget impossib
le.
pub fn recover_on_startup(cat: &Catalog) -> AuraResult<Resume
Summary> {
    let summary = cat.writer().transact(move |tx| {
        let now = crate::now(tx)?;
        let orphaned: i64 = tx.query_row(
            "SELECT COUNT(*) FROM task WHERE state = 'runnin

```



```

g'", [], |r| r.get(0)
).map_err(crate::errors::store_write)?;

tx.execute(
    "UPDATE task
      SET state = CASE WHEN attempts >= max_attempts
        THEN 'failed' ELSE 'ready' END,
        error_code = CASE WHEN attempts >= max_attempts
          THEN 'AURA-JOB-7002' ELSE 'AURA-JOB-7001' END,
        lease_owner = NULL, lease_expires_at = NULL, heartbeat_at = NULL,
        updated_at = ?1
      WHERE state = 'running'",
    rusqlite::params![now],
).map_err(crate::errors::store_write)?;

// Consistency audit. If any of these hold, the catalog contradicts
// itself and continuing would produce wrong output silently.
let bad_pending: i64 = tx.query_row(
    "SELECT COUNT(*) FROM task t WHERE t.state = 'pending'
      AND NOT EXISTS (SELECT 1 FROM task_dep d WHERE d.task_id = t.task_id)",
    [], |r| r.get(0)).map_err(crate::errors::store_write)?;

let orphan_deps: i64 = tx.query_row(
    "SELECT COUNT(*) FROM task_dep d
      WHERE NOT EXISTS (SELECT 1 FROM task WHERE task_id = d.depends_on)",
    [], |r| r.get(0)).map_err(crate::errors::store_write)?;

if bad_pending > 0 || orphan_deps > 0 {

```

```

        return Err(crate::errors::inconsistent_resume_state(
            bad_pending, orphan_deps));
    }

    // Also promote anything whose dependencies completed
    // before the crash
    // but whose promotion transaction never committed. Belt
    // and braces:
    // promotion is already transactional, but this costs
    // one statement and
    // removes a whole class of stuck-run tickets.
    tx.execute(
        "UPDATE task SET state = 'ready', updated_at = ?1
        WHERE state = 'pending' AND NOT EXISTS (
            SELECT 1 FROM task_dep d JOIN task t2 ON t2.task_id = d.depends_on
            WHERE d.task_id = task.task_id AND t2.state <> 'succeeded')",
        rusqlite::params![now],
    ).map_err(crate::errors::store_write)?;

    crate::store::count_states(tx, orphaned)
    })?;

    tracing::info!(target: "jobs",
        orphaned = summary.orphaned_leases, ready = summary.ready,
        succeeded = summary.succeeded, failed = summary.failed,
        "resume complete");
    Ok(summary)
}

#[derive(Debug, Clone, Default)]
pub struct ResumeSummary {
    pub orphaned_leases: i64,

```

```

    pub ready: i64,
    pub pending: i64,
    pub succeeded: i64,
    pub failed: i64,
    pub quarantined: i64,
    pub skipped: i64,
}

```

4. Worker loop — `crates/aura-jobs/src/worker.rs`

```

use std::sync::{atomic::{AtomicBool, Ordering}, Arc};

use aura_catalog::Catalog;
use aura_core::AuraResult;

use crate::contract::graph::*;

/// What a stage actually does. Phases 02 onward implement th
is trait and
/// register it; the worker loop never changes again.
pub trait StageHandler: Send + Sync {
    fn stage(&self) -> &'static str;
    fn stage_version(&self) -> u32;
    /// Must be cancellation-aware: check `cancel` at least e
very 250 ms so the
    /// 500 ms cancel budget is achievable.
    fn execute(&self, cat: &Catalog, task: &Task, cancel: &Ca
ncelToken) -> AuraResult<()>;
}

pub struct Worker {
    id: WorkerId,
    cat: Arc<Catalog>,
    handlers: Vec<Arc<dyn StageHandler>>,
    budget: CostHint,
}

```

```

        cancel: CancelToken,
    }

impl Worker {
    /// One iteration: claim, run with a heartbeat, complete.
    Returns false when
    /// there is nothing left to claim, so the caller can idle
    instead of spin.
    pub fn step(&self) -> AuraResult<bool> {
        let Some(task) = crate::store::claim_next(&self.cat,
self.project, &self.id, self.budget)? else {
            return Ok(false);
        };

        let Some(handler) = self.handlers.iter().find(|h| h.stage() == task.stage) else {
            // An unknown stage is a programming error, not a
            data error. Fail
            // loudly with a code rather than silently leaving
            the task Ready to
            // be claimed again in an infinite loop.
            crate::store::complete(&self.cat, &task.task_id,
&self.id,
                Outcome::Failed(crate::errors::unknown_stage
(&task.stage)))?;
            return Ok(true);
        };

        // Heartbeat thread. If it discovers the lease was re
        claimed it sets
        // `lost`, which the handler observes through the can
        cel token: the
        // worker stops doing work nobody is waiting for.
        let lost = Arc::new(AtomicBool::new(false));
        let hb = self.spawn_heartbeat(&task.task_id, Arc::clone(&lost));

```

```

        let result = handler.execute(&self.cat, &task, &self.
cancel);
        hb.stop();

        if lost.load(Ordering::SeqCst) {
            tracing::warn!(target: "jobs", task = %task.task_
id, "lease lost, discarding result");
            return Ok(true);
        }

        let outcome = match result {
            Ok(()) => Outcome::Succeeded,
            Err(e) if e.recovery == aura_core::Recovery::Quar
antine => Outcome::Quarantined(e),
            Err(e) if task.attempts >= task.max_attempts => 0
utcome::Failed(e),
            Err(e) => Outcome::Retry(e),
        };
        crate::store::complete(&self.cat, &task.task_id, &sel
f.id, outcome)?;
        Ok(true)
    }
}

```

Decision	Alternative rejected	Why
Deterministic <code>TaskId</code>	Random UUID per task	Re-enqueue on resume would duplicate every task in the graph
Lease TTL in the database	In-memory worker registry	A registry dies with the process; the whole point is surviving that
<code>attempts</code> incremented at claim	Increment on failure	A worker killed before it can report a failure would retry for ever
Startup reclaim ignores expiry	Wait for leases to lapse	90 s wait versus a 2 s resume budget

Decision	Alternative rejected	Why
Promotion in the completion transaction	Separate scheduler pass	A crash between the two leaves dependents stuck in Pending for ever
Consistency audit on startup	Trust the rows	AURA-JOB-7005 beats producing a wrong album from a wrong graph
Result discarded if lease lost	Write it anyway	Two workers writing the same result is how double-counting starts

5. Phase-01 stage graph

Phase 01 registers exactly three stages. They are deliberately trivial — the point is to prove the machinery, not to do clever work.

```

scan:{import_id}:1
  | (one task per import, produces the file list)
  ▼
hash:{file_id}:1      — one task per file, parallel, cost_ram_mb = 8
  |
  ▼
index:{photo_id}:1    — one task per photo: EXIF + catalog rows

```

```

// crates/aura-jobs/src/stages/mod.rs
pub const STAGE_SCAN: &str = "scan";
pub const STAGE_HASH: &str = "hash";
pub const STAGE_INDEX: &str = "index";

// Priorities: lower runs first. Scan must finish before hash
// ing is useful,
// and indexing a photo is what makes it appear in the grid,
// so it outranks
// hashing the next batch. Photographers watch the grid fill;
// they do not
// watch a hash counter.
pub const PRIORITY_SCAN: i32 = 10;

```

```
pub const PRIORITY_INDEX: i32 = 20;
pub const PRIORITY_HASH: i32 = 30;
```

6. Tests for T11

```
// crates/aura-jobs/tests/graph.rs

#[test]
fn enqueue_is_idempotent() {
    let (cat, specs) = fixture_graph(500);
    assert_eq!(enqueue(&cat, prj(), run(), &specs).expect("first"), 500);
    assert_eq!(enqueue(&cat, prj(), run(), &specs).expect("second"), 0);
    assert_eq!(cat.count("task").expect("count"), 500);
    assert_eq!(cat.count("task_dep").expect("count"), expected_edges());
}

#[test]
fn dependency_on_unknown_task_is_rejected_before_any_write() {
    let spec = TaskSpec { depends_on: vec![TaskId::new("ghost", "x", 1).expect("id")], ..basic() };
    let err = enqueue(&cat, prj(), run(), &[spec]).err().expect("must reject");
    assert_eq!(err.code.0, "AURA-JOB-7003");
    assert_eq!(cat.count("task").expect("count"), 0, "nothing may be written");
}

#[test]
fn illegal_transitions_are_impossible() {
    for from in ALL_STATES { for to in ALL_STATES {
        if !from.can_transition_to(to) {
```

```

        assert!(force_transition(from, to).is_err(), "{from:?} -> {to:?} must be rejected");
    }
}

#[test]
fn dependents_become_ready_only_when_all_deps_succeed() {
    // Diamond: a -> {b, c} -> d. d is Ready only after both
    b and c succeed.
}

#[test]
fn failed_task_skips_all_transitive_dependents_exactly_once()
{
    // Diamond again: fail a, assert b, c, d are Skipped and
    d is visited once.
}

#[test]
fn quarantined_task_does_not_skip_unrelated_work() {
    let ids = enqueue_two_independent_chains();
    quarantine(&ids.chain_a_first);
    assert_eq!(state(&ids.chain_b_first), TaskState::Ready);
}

#[test]
fn expired_lease_is_reclaimed_and_retried() {
    let cat = fresh_with_fixed_clock();
    let task = claim_one(&cat);
    cat.clock().advance_ms(91_000); // FixedClock, n
    o sleeping
    assert_eq!(reclaim_expired(&cat).expect("reclaim"), 1);
    assert_eq!(state(&task), TaskState::Ready);
    assert_eq!(attempts(&task), 1);
}

```



```

#[test]
fn task_failing_max_attempts_times_lands_in_failed_not_a_loop
() {
    let task = claim_one(&cat);
    for _ in 0..DEFAULT_MAX_ATTEMPTS { expire_and_reclaim(&cat, &task); }
    assert_eq!(state(&task), TaskState::Failed);
    assert_eq!(error_code(&task), Some("AURA-JOB-7002".to_string()));
}

#[test]
fn heartbeat_after_reclamation_returns_false_and_result_is_discarded() {
    let task = claim_one_as(&cat, &worker_a());
    cat.clock().advance_ms(91_000);
    reclaim_expired(&cat).expect("reclaim");
    assert!(!heartbeat(&cat, &task, &worker_a()).expect("heartbeat"));
    let err = complete(&cat, &task, &worker_a(), Outcome::Succeeded).err().expect("must fail");
    assert_eq!(err.code.0, "AURA-JOB-7004");
}

#[test]
fn startup_recovery_returns_running_tasks_to_ready_immediately() {
    let cat = fresh();
    let _t = claim_one(&cat); // lease is live, not expired
    let summary = recover_on_startup(&cat).expect("recover");
    assert_eq!(summary.orphaned_leases, 1);
    assert_eq!(summary.ready, 1);
}

```

```

#[test]
fn inconsistent_graph_is_refused_with_7005() {
    let cat = fresh();
    corrupt_graph_by_deleting_a_dependency_row(&cat);
    let err = recover_on_startup(&cat).err().expect("must ref
use");
    assert_eq!(err.code.0, "AURA-JOB-7005");
}

#[test]
fn claim_order_is_deterministic_under_equal_priority_and_time
stamp() {
    let ids = enqueue_50_identical_priority_same_millisecond
();
    let order_a: Vec<_> = drain_claim_order(&fresh_from(&id
s));
    let order_b: Vec<_> = drain_claim_order(&fresh_from(&id
s));
    assert_eq!(order_a, order_b);
}

#[test]
fn cost_budget_prevents_claiming_a_task_that_would_not_fit()
{
    enqueue_task_with(CostHint { vram_mb: 6000, ..Default::de
fault() });
    let claimed = claim_next(&cat, prj(), &worker_a(), CostHi
nt { vram_mb: 2000, ..Default::default() });
    assert!(claimed.expect("call").is_none(), "must not claim
a task that does not fit");
}

```

Chaos test: the one that matters

```

// crates/aura-jobs/tests/chaos_resume.rs

/// Kill the process at eight points during a 4,000-file import,
/// relaunch, and
/// assert the catalog converges to exactly the same state as
/// an uninterrupted
/// run. This test is the reason the whole crate exists.
#[test]
#[ignore = "chaos: run in the nightly lane, spawns child processes"]
fn kill_at_every_stage_boundary_converges_to_the_same_catalog
() {
    let reference = run_import_uninterrupted(); // digest
    of a clean run

    for kill_point in KILL_POINTS { // 8 named points
        let dir = fresh_fixture_copy();
        let child = spawn_import_child(&dir, kill_point);
        child.wait_until_signal(kill_point);
        child.kill_9(); // SIGKILL, no cleanup

        let t0 = std::time::Instant::now();
        let cat = open_catalog(&dir).expect("catalog must open after kill -9");
        let summary = recover_on_startup(&cat).expect("resume");
        let resume_ms = t0.elapsed().as_millis();

        assert!(resume_ms < 5_000, "resume took {resume_ms} ms at {kill_point:?}");
        run_import_to_completion(&cat);

        assert_eq!(cat.catalog_digest().expect("digest"), ref

```

```

erence,
    "catalog diverged after kill at {kill_point:?}");
    assert_eq!(count_state(&cat, TaskState::Running), 0,
    "no task may remain Running");
    assert!(summary.orphaned_leases <= 8, "at most one lease per worker");
    cat.integrity_check().expect("integrity after kill");
}
}

const KILL_POINTS: &[KillPoint] = &[
    KillPoint::AfterScanBeforeFirstHash,
    KillPoint::MidHashBatch,
    KillPoint::BetweenHashAndIndex,
    KillPoint::MidInsertTransaction,
    KillPoint::AfterCommitBeforeCounterUpdate,
    KillPoint::DuringHeartbeat,
    KillPoint::DuringWalCheckpoint,
    KillPoint::DuringMigration,
];

```

7. Acceptance criteria for T11–T12

- ☐ `enqueue` of 500 specs twice inserts 500 then 0; edge count unchanged
- ☐ A dependency on an unknown task is rejected with `AURA-JOB-7003` and writes nothing
- ☐ Exhaustive transition test passes: every illegal `(from, to)` pair is rejected by the store, not merely by the enum
- ☐ Diamond graph promotes `d` only after both `b` and `c` succeed
- ☐ A `Failed` task marks every transitive dependent `Skipped`, each visited once
- ☐ A `Quarantined` task leaves unrelated chains untouched
- ☐ Expired lease returns the task to `Ready` with `AURA-JOB-7001`; the third expiry yields `Failed` with `AURA-JOB-7002`

- ☐ `heartbeat` returns false after reclamation and `complete` then fails with `AURA-JOB-7004`; no result is written
- ☐ Startup recovery returns live-lease `Running` tasks to `Ready` without waiting for expiry
- ☐ A deliberately corrupted graph is refused with `AURA-JOB-7005`
- ☐ Claim order is identical across two runs with equal priorities and timestamps
- ☐ Cost budget prevents claiming a task that does not fit
- ☐ All eight chaos kill points converge to the reference catalog digest; timings pasted, all under 5 s
- ☐ No task remains `Running` after any kill; `integrity_check` passes in all eight cases
- ☐ Every test uses `FixedClock`; `grep -rn "thread::sleep" crates/aura-jobs/tests` returns nothing except the chaos child process

8. Claude Code prompt for T11–T12

Act as `SRC` (Senior Rust, Core), reviewing under the `QAIQ` hat. Implement T11 and T12 of PHASE-01 from this page. Copy `contract/graph.rs` **verbatim**, including every doc comment — the comments record decisions I do not want re-litigated.

Then implement `store.rs`, `reclaim.rs`, `worker.rs`, `stages/mod.rs`, and every test in section 6 including the four sketched as comments and the chaos test in full. The chaos test spawns a real child process and sends a real `SIGKILL`; do not simulate it with a flag.

Hard constraints: `claim_next` must be one statement with `RETURNING`; `attempts` increments at claim time; promotion and skipping happen inside the completion transaction; a worker that has lost its lease must not write a result; startup recovery must not wait for lease expiry; every ordering used for claiming must be total and deterministic; time comes only from the injected `Clock`, and no test may call `thread::sleep` to wait for a lease.

Evidence I expect pasted back: `cargo test -p aura-jobs` in full; the chaos test output listing all eight kill points with resume timings and matching digests; and

| the state-transition matrix test output. Stop after T12.